

Tansu: decentralized project governance for open-source software on Stellar

Pamphile T. Roy

Consulting Manao GmbH, Austria

ORCID: 0000-0001-9816-1416

July 2026

Abstract

Open-source projects still lack an auditable, community-weighted record of which commits are *released* or *revoked*. CVE-2024-3094 showed maintainer capture can beat pipeline attestation [8, 16]; TUF, in-toto, Sigstore, and SLSA trace artefacts to source revisions [18, 33, 10, 12] but record no equivalent governance outcome. Tansu closes that gap with Soroban contracts on Stellar that store maintainer lists, commit hashes, weighted outcomes, and IPFS pointers to SBOMs, CVE reports, and attestations keyed by commit. Ballots may be public or hidden behind Pedersen commitments over BLS12-381, with the weighted tally verified on-chain. Members may bind a Git SSH key to their Stellar account at registration. On Stellar mainnet the contracts administer the Public Goods Award; anonymous mode still depends on a designated tallying party for ballot secrecy.

Keywords: decentralized governance; open-source software; software supply chain; anonymous voting; smart contracts; Stellar

1 Introduction

Most software still pulls code from a central forge. Compromise the maintainer account or CI job and you can ship malicious releases without touching the provenance tooling downstream users trust. Ohm et al. counted 174 malicious packages on npm, PyPI, and RubyGems over four years [11]; Ladisa et al. later listed 107 attack vectors across the open-source supply chain [6]. The 2024 XZ Utils incident is a sharper example: the attacker spent years earning maintainer access and only then planted the backdoor [8, 16].

Artifact signing and pipeline attestation are necessary, but the XZ attack exploited maintainer trust rather than missing signatures. Section 2 surveys TUF, in-toto, Sigstore, and SLSA; none stores a project vote that marks a commit as *released* or *revoked*.

A separate risk is operational. Sigstore entries are published through operated services such as Rekor and Fulcio [10, 23]; TUF deployments rely on maintained metadata mirrors; the global CVE namespace is curated under a U.S. government contract with MITRE [9]. In April 2025 MITRE warned that CVE program funding was about to lapse, which would have halted new assignments until CISA granted a short extension [2]. Neither case is on-chain fraud, but both show how much of the stack can pause when a single operator or sponsor falters. Tansu does not replace those registries; it records project votes and evidence pointers on a public ledger where, once written, the anchors are replicated with the chain rather than held behind one vendor’s uptime.

Tansu fills that gap with Soroban contracts written in Rust [4]. Source code remains on conventional git forges (GitHub, GitLab, Codeberg, Gitea, Bitbucket, Radicle, and others); the ledger stores registrations, maintainer lists, commit hashes, proposals, votes, and outcomes that

Table 1: Comparison with adjacent provenance and governance systems. Provenance frameworks trace build artefacts; governance platforms record votes. Sigstore, TUF mirrors, and the CVE program depend on operated registries whose uptime is not guaranteed [23, 2]. Tansu combines on-chain governance with commit-keyed evidence anchors.

System	On-chain governance	Commit anchor	Evidence per commit	Private ballots
TUF [18]	–	–	–	–
in-toto [33]	–	partial	attestations	–
Sigstore [10]	–	partial	signatures	–
Snapshot [24]	off-chain	–	–	–
OpenZeppelin Governor [13]	yes	–	–	–
Tansu	yes	yes	yes	optional

any observer can read from public ledger data. We describe contract layout, anonymous voting, commit-anchored evidence, and the Public Goods Award deployment on mainnet.

Within those contracts, code finality attaches trust states to commits through weighted votes; maintainers record SBOM, CVE, and attestation CIDs per commit hash; anonymous voting hides individual choices behind Pedersen commitments while verifying the aggregate tally on-chain; and member registration checks an SSH signature against SEP-53. The contracts are deployed on Stellar mainnet [31].

2 Related work

Supply-chain provenance. The Update Framework introduced survivable key compromise for software update systems [18] and underpins several package managers. in-toto extends provenance to whole pipelines by having each step sign metadata about its inputs and outputs [33]. Sigstore removes long-lived signing keys by binding ephemeral certificates to OpenID identities [10], and SLSA defines provenance levels that build systems can claim and verify [12]. In practice, consumers still reach much of this material through operated registries: Sigstore’s transparency log [23], TUF metadata hosts, and the CVE program run by MITRE under federal contract [9, 2]. When those services pause or their funding is uncertain, downstream tools lose a shared reference even though the underlying signatures may still exist offline.

Cryptographic voting. Helios demonstrated practical open-audit elections on the web [1]. The Open Vote Network implemented a self-tallying, maximally private boardroom protocol as an Ethereum contract [7]. Both target a different deployment model than a live governance contract with badge-weighted votes and collateral deposits.

On-chain governance. Snapshot provides off-chain, gasless voting for token communities [24], and OpenZeppelin’s Governor contracts are widely used for on-chain DAO governance on Ethereum [13]. Fritsch et al. found voting power in major DAOs concentrated in a handful of addresses [5]. Spychiger et al. ran a controlled DAO experiment for project management [25].

Tansu sits between the last two groups. Governance runs on-chain, and a passed proposal can write a commit hash into contract state. Weights come from maintainer badges or Neural Quorum Governance scores rather than token balances. Evidence and votes are indexed by commit alongside the provenance artefacts that TUF, in-toto, Sigstore, and SLSA help produce. Table 1 summarises how Tansu differs from adjacent governance and provenance systems.

Table 2: Badge types and their voting weights in the main contract.

Badge	Weight
Developer	10,000,000
Triage	5,000,000
Community	1,000,000
Verified	500,000
Default (no badge)	1

3 System overview

Tansu consists of a main governance contract and an auxiliary membership contract, both written in Rust against the Soroban SDK, plus a web client, command-line tooling, and continuous-integration recipes. The code is released under the BSD 3-Clause License [17].

Architecture. The forge stays the forge: Git history, pull requests, and review threads are not copied to the chain. On-chain state is limited to the facts a third party needs to audit governance—maintainer addresses, the current commit hash, proposal metadata, vote weights, and results. Proposal bodies, member profiles, and evidence files reside on IPFS while the contract stores only their CIDs. Each state transition emits an event, so indexers can reconstruct the full history even when the contract retains only the latest value, such as the current commit hash.

Projects and commits. A project registers under a short name; the contract derives its key as the Keccak-256 hash of the name and stores the maintainer set, the forge URL, and the CID of a metadata file. Registration requires a collateral deposit, which deters name squatting. Any maintainer can then update the project’s current commit hash. The contract keeps only the latest hash and emits an event per update, so the full sequence of attested hashes remains recoverable from the event stream.

Membership and voting weight. Members register themselves with a profile CID. Maintainers assign per-project badges that determine the maximum voting weight of a member (Table 2). Alternatively, a proposal can be token-based, in which case weight equals the amount of a designated token the voter locks for the duration of the vote. A third mode delegates weights to an external contract: for the Stellar Community Fund deployment, weights come from Neural Quorum Governance (NQG) scores queried live from the fund’s scoring contract, with a minimum threshold that restricts voting to community pilots [21]. A separate soulbound NFT contract (SEP-50 [29]) exposes each member’s role and NQG score as token traits.

Proposals and code finality. A proposal carries a title, the CID of its full text, a voting deadline, and optionally a list of *outcome contracts*: cross-contract calls executed when the proposal is approved, rejected, or cancelled. Code finality uses this hook. A project defines trust states for its releases (for example *released*, *security-reviewed*, or *revoked*) and records each state change against a commit hash through a proposal whose outcome contract writes the new state. The decision rule is a supermajority. With weighted tallies T_a , T_r , and T_{ab} for approve, reject, and abstain, a proposal passes only if $T_a > T_r + T_{ab}$, is rejected only if $T_r > T_a + T_{ab}$, and is cancelled otherwise. Execution waits for a timelock after the voting deadline. Proposal creation and voting require collateral deposits refunded on execution; maintainers may revoke a malicious proposal or remove an abusive vote and withhold the collateral. They may also bar addresses with a declared conflict of interest from voting on a specific proposal.

4 Commit-anchored evidence

SBOMs, CVE write-ups, and attestation bundles still reach users through release pages, mailing lists, and vendor portals, where they are scattered and difficult to audit after publication. Central registries compound the problem: if Sigstore’s log or the CVE feed is unreachable, consumers lose the default discovery path even when a project already has the document. Tansu assigns each project-maintained document a fixed on-chain slot keyed by the commit it describes. The slot stores a CID and timestamp on the ledger; anyone can read that anchor from chain history without asking a particular operator to stay online.

The contract exposes two functions. A maintainer calls `set_evidence` with a project key, a commit hash, an evidence kind, and a CID; anyone calls `get_evidence` to retrieve the recorded history for a given commit and kind. Three kinds are supported: `Sbom` for software bills of materials (e.g., CycloneDX [14] or SPDX [32] documents), `Cve` for vulnerability reports referencing CVE identifiers [9], and `Attestation` for build attestations such as SLSA provenance bundles [12]. The evidence itself lives on IPFS; the contract stores only the CID and a timestamp.

Evidence is append-only. Each call adds an entry to the history for the triple (project, commit, kind) rather than overwriting it, so successive vulnerability re-scans of the same commit accumulate: a commit that was clean at release time and gained a CVE report two years later carries both records. The contract keeps the most recent ten entries per triple; older entries roll off on-chain but remain recoverable from the emitted events, which indexers retain in full.

In our own repository, a continuous-integration workflow generates an SBOM on every release tag, uploads it to IPFS, and records the CID on-chain in the same run. A single query against the contract and an IPFS gateway returns the release commit, its SBOM, and any posted attestations without depending on the forge remaining available or trustworthy.

The contract records who published each entry and when; only maintainers can write. It does not parse SBOM syntax, check CVE applicability, or walk an attestation chain, and it does not require the commit to be the project’s current head, so historical commits remain valid targets. Consumers perform that validation off-chain. On-chain, the contract provides an append-only, tamper-evident log of what the project asserted about each commit: later maintainers cannot silently rewrite an earlier entry, though they can add new ones.

5 Anonymous voting

Public voting is the default and requires no cryptography: votes are posted in the clear and tallied by the contract. Funding rounds and personnel votes in the Public Goods Award required ballot secrecy that public tallies could not provide. Tansu’s anonymous mode never publishes individual choices, yet the aggregate tally is still verified on-chain.

Setup. A maintainer configures anonymous voting once per project. The contract derives two generators G and H of the BLS12-381 $\mathbb{G}_1$ group by hashing two fixed labels to the curve under distinct domain-separation tags. No discrete-logarithm relation between G and H is known to any party, and no trusted setup is required. The maintainer also registers a project-level RSA-OAEP public key (2048-bit, SHA-256) used to encrypt ballot payloads client-side.

Ballots. A ballot is a one-hot vector $v = (v_a, v_r, v_{ab}) \in \{0, 1\}^3$ over the choices approve, reject, abstain. For each component i the voter samples a random seed r_i and computes a Pedersen commitment [15]

$$C_i = v_i G + r_i H. \tag{1}$$

The voter submits the three commitments together with RSA-OAEP ciphertexts of v and r under the project public key. The contract checks that each C_i is a valid point in the prime-order subgroup of $\mathbb{G}_1$, stores the ciphertexts without processing them, and folds the weighted commitments into running aggregates

$$A_i \leftarrow A_i + w C_i, \quad (2)$$

where w is the voter’s weight at the time of the vote. Commitments are computed client-side; the contract offers the same computation as a read-only simulation helper, which must never be submitted as a transaction.

Tally and verification. After the deadline, the tallying party (a maintainer holding the decryption key) decrypts all ballots and computes the weighted sums $T_i = \sum_j w_j v_{j,i}$ and $S_i = \sum_j w_j r_{j,i}$. It submits (T, S) to the contract, which accepts the tally if and only if

$$T_i G + S_i H = A_i \quad \text{for } i \in \{a, r, ab\}. \quad (3)$$

By the homomorphism of (1), equation (3) holds exactly when the submitted tallies are consistent with every recorded ballot; under the discrete-logarithm assumption the commitments are binding, so the tallying party cannot substitute a different result. The seeds provide information-theoretic hiding, so the commitments reveal nothing about individual choices. The supermajority rule of Section 3 is then applied to T .

Trust model. Ballot privacy is not trustless in this mode. The tallying key holder can decrypt individual votes and can stall execution by refusing to submit (T, S) ; lost keys freeze a proposal. The chain nonetheless enforces that any announced tally matches the recorded commitments, which a trusted tallier without cryptographic checks could falsify. In practice we rotate who holds the key, and public voting remains available when the threat model permits it.

6 Git identity binding

Votes count on Stellar addresses while patches land under Git handles. Members bind the two during `add_member` or `update_member` by submitting a forge handle, an Ed25519 public key, a challenge message, and an SSH signature over that message per SEP-53 [30]. The contract invokes the native Ed25519 verifier; on success it stores the handle and key and emits an event.

The signature proves key possession at registration time. Matching the key to the handle is left to clients: GitHub publishes `github.com/<user>.keys`, GitLab provides an equivalent endpoint, and Radicle aliases in `tansu.toml` can be checked against repository metadata. Anyone can compare the result. The contract does not call a forge API.

7 Deployment and adoption

Tansu runs on Stellar mainnet [31]. The contract reports version 2; upgrades are gated by an M-of-N admin threshold and a timelock. Development was funded through the Stellar Community Fund’s Activation and Build tracks [26, 27].

Its main production workload is the Public Goods Award: proposal submission, NQG-weighted voting, and outcome execution on Tansu [20, 22, 21]. Contract changes for that process were funded separately [28]. The project also participates in Drips Wave [3], a recurring funding programme for open-source contributors, and each release tag in our repository triggers the evidence workflow of Section 4.

Table 3: Public Goods Award outcomes (verified from intake records in the working-group repository [19]).

Metric	Value
Funded projects	17
Total disbursed	~USD 400,000
Governance contract	mainnet v2

7.1 Evaluation

The mainnet contract address is listed on Stellar Expert [31]; `version()` returns 2. Contract source, client code, and integration tests are available in the public repository [17]. Proposals, votes, evidence entries, and commit updates each emit events, so the history is reconstructible from ledger archives without privileged access.

The Public Goods Award is the primary production workload on that contract. Table 3 reports round outcomes verified against intake records in the working-group repository [19, 20]. Seventeen projects received funding; total disbursement was approximately USD 400,000. Individual proposal text, vote tallies, and execution transactions remain on-chain and in the archived intake forms cited above.

8 Limitations

Anonymous voting trusts the tallying party for ballot secrecy (Section 5). Commit hashes reflect whatever maintainers post; a dishonest quorum could misstate the current head, and the contract would not detect the discrepancy unless a client compared against the forge. Evidence CIDs are not validated, so a maintainer can anchor arbitrary content. Bulk files still depend on IPFS pinning; the chain records pointers, not bytes, and does not replace the CVE program or Sigstore—it complements them when operated registries are slow or unavailable. Badge assignment, NQG scores, and NFT admission all sit upstream of the contracts: the chain records who granted whom weight, not whether that weight was justified.

9 Conclusion

Tansu records on Stellar what a forge cannot: weighted votes, commit pointers, and evidence references tied to specific revisions. The Public Goods Award deployment funded seventeen projects for roughly USD 400,000, with intake records published in the working-group repository [19]. Provenance frameworks trace build artefacts to source; Tansu adds a governance vote and document index on the same commit hash. Open items include threshold decryption for anonymous voting, more evidence kinds, and finer-grained turnout statistics.

Acknowledgements

Funded by the Stellar Community Fund (Activation and Build tracks). Thanks to the Public Goods Award working group for running the first production workload on the contracts. Parts of the web client and this manuscript were edited with generative AI tools; the smart contracts were not.

References

- [1] Ben Adida. Helios: Web-based open-audit voting. In *Proceedings of the 17th USENIX Security Symposium (USENIX Security 08)*, pages 335–348. USENIX Association, 2008. URL <https://www.usenix.org/conference/17th-usenix-security-symposium/helios-web-based-open-audit-voting>.
- [2] Christopher Bing. U.s. funding running out for critical cyber vulnerability database, manager says. Reuters, April 2025. URL <https://www.reuters.com/technology/us-funding-running-out-critical-cyber-vulnerability-database-manager-says-2025-04-15/>. Reporting on the April 2025 CVE program funding lapse.
- [3] Drips Network. Drips wave, 2026. URL <https://docs.drips.network/wave/>.
- [4] Stellar Development Foundation. Soroban smart contracts documentation, 2026. URL <https://developers.stellar.org/docs/build/smart-contracts>.
- [5] Robin Fritsch, Marino Müller, and Roger Wattenhofer. Analyzing voting power in decentralized governance: Who controls DAOs? *Blockchain: Research and Applications*, 5(3):100208, 2024. doi: 10.1016/j.bcr.2024.100208. URL <https://doi.org/10.1016/j.bcr.2024.100208>.
- [6] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. Sok: Taxonomy of attacks on open-source software supply chains. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 1509–1526. IEEE, 2023. doi: 10.1109/SP46215.2023.10179304. URL <https://doi.org/10.1109/SP46215.2023.10179304>.
- [7] Patrick McCorry, Siamak F. Shahandashti, and Feng Hao. A smart contract for boardroom voting with maximum voter privacy. In *Financial Cryptography and Data Security (FC 2017)*, volume 10322 of *Lecture Notes in Computer Science*, pages 357–375. Springer, 2017. doi: 10.1007/978-3-319-70972-7_20. URL <https://eprint.iacr.org/2017/110>.
- [8] MITRE Corporation. CVE-2024-3094: malicious code in XZ Utils 5.6.0 and 5.6.1 tarballs, 2024. URL <https://www.cve.org/CVERecord?id=CVE-2024-3094>.
- [9] MITRE Corporation. Common vulnerabilities and exposures (CVE) program, 2026. URL <https://www.cve.org/>.
- [10] Zachary Newman, John Speed Meyers, and Santiago Torres-Arias. Sigstore: Software signing for everybody. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS ’22)*, pages 2353–2367. ACM, 2022. doi: 10.1145/3548606.3560596. URL <https://doi.org/10.1145/3548606.3560596>.
- [11] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. Backstabber’s knife collection: A review of open source software supply chain attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA 2020)*, volume 12223 of *Lecture Notes in Computer Science*, pages 23–43. Springer, 2020. doi: 10.1007/978-3-030-52683-2_2. URL https://doi.org/10.1007/978-3-030-52683-2_2.
- [12] Open Source Security Foundation (OpenSSF). Supply-chain levels for software artifacts (slsa), 2026. URL <https://slsa.dev/>.
- [13] OpenZeppelin. Governor smart contracts, 2026. URL <https://docs.openzeppelin.com/contracts/5.x/governance>.

- [14] OWASP Foundation. CycloneDX bill of materials specification, 2026. URL <https://cyclonedx.org/>.
- [15] Torben Pryds Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *Advances in Cryptology — CRYPTO '91*, volume 576 of *Lecture Notes in Computer Science*, pages 129–140. Springer, 1992. doi: 10.1007/3-540-46766-1_9. URL https://doi.org/10.1007/3-540-46766-1_9.
- [16] Piotr Przymus and Thomas Durieux. Wolves in the repository: A software engineering analysis of the XZ Utils supply chain attack. In *Proceedings of the 22nd International Conference on Mining Software Repositories (MSR 2025)*. IEEE/ACM, 2025. Also available as arXiv:2504.17473.
- [17] Pamphile Roy and Consulting Manao contributors. Tansu, 2026. URL <https://github.com/Consulting-Manao/tansu>. Software repository.
- [18] Justin Samuel, Nick Mathewson, Justin Cappos, and Roger Dingledine. Survivable key compromise in software update systems. In *Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS '10)*, pages 61–72. ACM, 2010. doi: 10.1145/1866307.1866314. URL <https://doi.org/10.1145/1866307.1866314>.
- [19] SCF Public Goods Maintenance Working Group. Scf public goods maintenance documentation and intake records, 2026. URL <https://github.com/SCF-Public-Goods-Maintenance/scf-public-goods-maintenance.github.io>. Public repository for Public Goods Award intake forms and process documentation.
- [20] SCF Public Goods Maintenance Working Group. Public goods award, 2026. URL <https://scf-public-goods-maintenance.github.io/public-goods-award/>.
- [21] SCF Public Goods Maintenance Working Group. Proposer instructions, 2026. URL <https://scf-public-goods-maintenance.github.io/pg-award/proposer-instructions/>.
- [22] SCF Public Goods Maintenance Working Group. Tansu modifications, 2026. URL <https://scf-public-goods-maintenance.github.io/pg-award/tansu-modifications/>.
- [23] Sigstore. Rekor transparency log overview, 2026. URL <https://docs.sigstore.dev/logging/overview/>. Operated transparency-log service for signed artefacts.
- [24] Snapshot Labs. Snapshot, 2026. URL <https://snapshot.org/>.
- [25] Florian Spychiger, Michael Lustenberger, Jens Martignoni, Lukas Schädler, and Patrick Lehner. Organizing projects with blockchain through a decentralized autonomous organization. *Project Leadership and Society*, 4:100102, 2023. doi: 10.1016/j.plas.2023.100102. URL <https://doi.org/10.1016/j.plas.2023.100102>.
- [26] Stellar Community Fund. Stellar community fund submission record (activation track), 2024. URL <https://communityfund.stellar.org/submissions/recheNDUSZjVVMowM>. Project submission record.
- [27] Stellar Community Fund. Stellar community fund submission record (build track), 2024. URL <https://communityfund.stellar.org/submissions/recSDU41kudJLzoXL>. Project submission record.
- [28] Stellar Community Fund. Stellar community fund submission record (public goods track), 2026. URL <https://communityfund.stellar.org/submissions/recC2cG06rZSYudtq>. Project submission record.

- [29] Stellar Development Foundation. SEP-50: Non-fungible tokens, 2024. URL <https://github.com/stellar/stellar-protocol/blob/master/ecosystem/sep-0050.md>. Stellar Ecosystem Proposal.
- [30] Stellar Development Foundation. SEP-53: Sign and verify messages, 2025. URL <https://github.com/stellar/stellar-protocol/blob/master/ecosystem/sep-0053.md>. Stellar Ecosystem Proposal.
- [31] StellarExpert. Tansu mainnet contract on stellar expert, 2026. URL <https://stellar.expert/explorer/public/contract/CDXINK2T3P46M4LWK35FVIXXHJ2XHAS4FOVCGVPJ63YV50VTM24IY5BI>. Public block-explorer record of the deployed contract.
- [32] The Linux Foundation. SPDX specification (ISO/IEC 5962:2021), 2021. URL <https://spdx.dev/>.
- [33] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 1393–1410. USENIX Association, 2019. URL <https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias>.